



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 06/10

NLP + IA du Chatbot

TF-IDF + Cosine + RAG + LLaMA + Memoire conversationnelle

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - C'est quoi le NLP ?	3
Chapitre 2 - Pipeline NLP complet	6
Chapitre 3 - Tokenisation	9
Chapitre 4 - Stopwords et stemming	12
Chapitre 5 - TF-IDF en details	15
Chapitre 6 - Cosine similarity	20
Chapitre 7 - n-grammes	24
Chapitre 8 - Detection de langue	27
Chapitre 9 - Multilingue (FR/EN/Darija)	30
Chapitre 10 - Memoire conversationnelle	33
Chapitre 11 - Personnalisation (genre/nom)	36
Chapitre 12 - LLM (LLaMA 3 + Groq)	39
Chapitre 13 - RAG = retrieval + generation	43
Chapitre 14 - Embeddings (vectorielle moderne)	47
Chapitre 15 - Conclusion	50

Chapitre 1 - C'est quoi le NLP ?

1.1 Definition

NLP = Natural Language Processing = Traitement Automatique du Langage Naturel. C'est le domaine de l'IA qui permet aux ordinateurs de **comprendre, analyser et generer du langage humain.**

■ **ANALOGIE :** Sans NLP, un ordinateur ne fait QUE du calcul mathematique. Avec NLP, il peut comprendre 'Bonjour, comment vas-tu ?' et repondre intelligemment. C'est ce qui fait la difference entre une calculatrice et Siri.

1.2 Applications du NLP

- **Assistants conversationnels :** ChatGPT, Siri, Alexa, notre chatbot FSBM
- **Traduction automatique :** Google Translate, DeepL
- **Analyse de sentiment :** detecter si un texte est positif/negatif
- **Resume automatique :** YouTube auto-resume, ChatGPT
- **Reconnaissance d'entites :** extraire noms, dates, lieux d'un texte
- **Classification de spam :** ton mail le fait deja
- **Detection de plagiat :** Turnitin, Compilatio
- **Recherche semantique :** Google Search comprend le sens, pas juste les mots

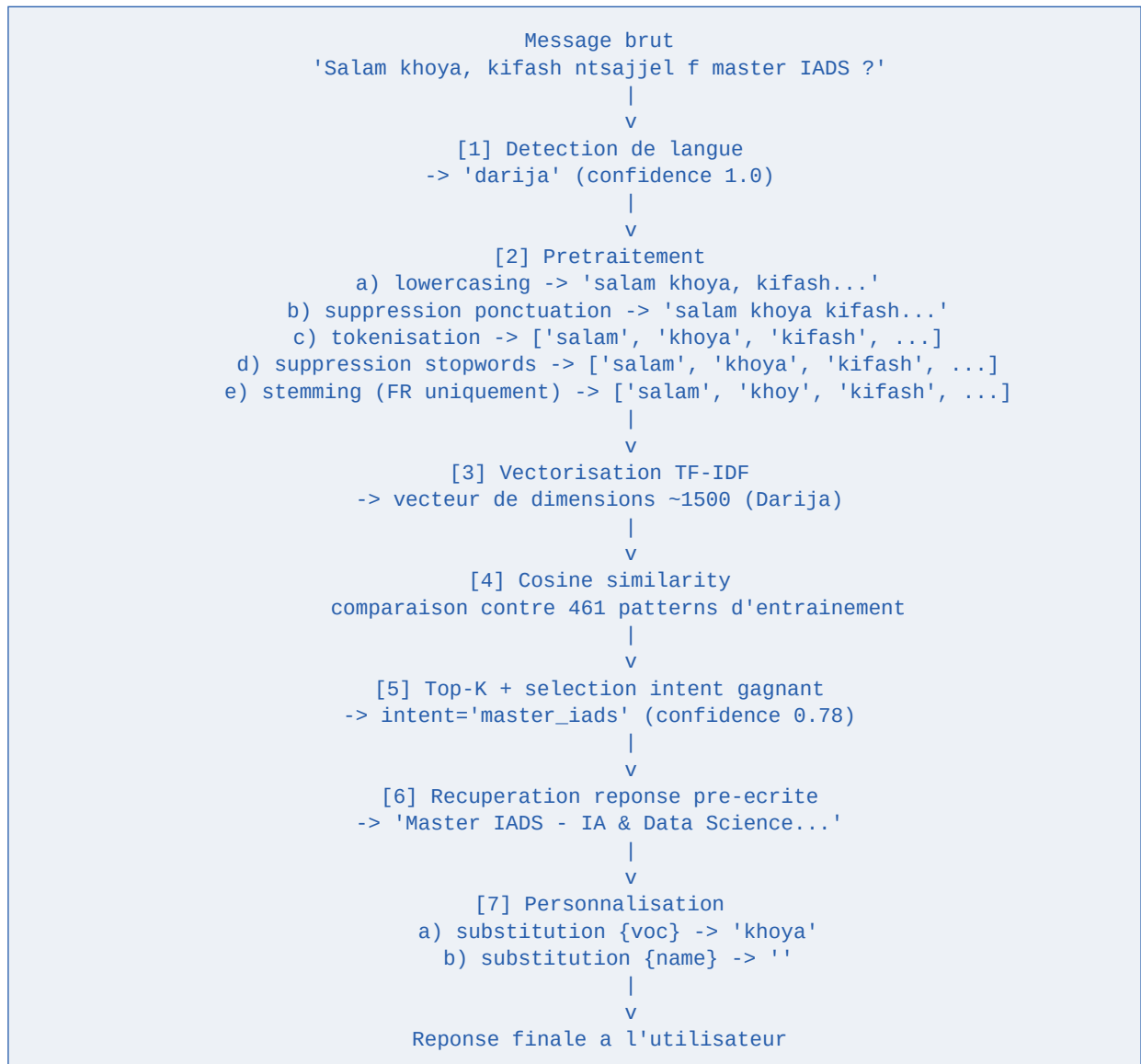
1.3 Les 2 ecoles

Approche	Methode	Exemples
Statistique	TF-IDF, regex, n-grammes	Notre chatbot v1
Deep learning	Reseaux de neurones, transformers	ChatGPT, BERT, LLaMA

[★] Notre projet combine les deux : **TF-IDF (statistique)** pour le retrieval rapide, **LLaMA 3 (deep learning)** pour la generation contextuelle.

Chapitre 2 - Pipeline NLP complet

2.1 Vue d'ensemble



2.2 Latences typiques

Etape	Temps
Detection langue	< 1 ms
Pretraitement	2-5 ms
Vectorisation TF-IDF	10-15 ms
Cosine similarity	20-30 ms
Selection top-K	< 1 ms
Personnalisation	< 1 ms
Total NLP	35-50 ms

Chapitre 3 - Tokenisation

3.1 C'est quoi un token ?

Un **token** est une **unite atomique** de texte. Pour notre NLP statistique, un token = un mot (apres nettoyage). Pour un LLM, c'est un sous-mot.

3.2 Exemple concret

```
Phrase : 'Bonjour, comment allez-vous ?'

Etape 1 : enlever la ponctuation
-> 'Bonjour comment allez vous'

Etape 2 : mettre en minuscules
-> 'bonjour comment allez vous'

Etape 3 : decouper en tokens (split sur espace)
-> ['bonjour', 'comment', 'allez', 'vous']

Total : 4 tokens
```

3.3 Notre implementation

```
# nlp/preprocessor.py
import re
import unicodedata

def tokenize(text: str) -> list[str]:
    text = text.lower()
    text = re.sub(r'[^\w\s]', ' ', text) # ponctuation -> espaces
    text = re.sub(r'\d+', ' ', text)    # nombres -> espaces
    return [t for t in text.split() if t] # split + remove vides

# Usage
tokens = tokenize("Bonjour, comment allez-vous ?")
# ['bonjour', 'comment', 'allez', 'vous']
```

3.4 Gestion des accents

Pour que 'eleve' et 'élève' soient consideres identiques :

```
def remove_accents(text: str) -> str:
    return ''.join(
        c for c in unicodedata.normalize('NFD', text)
        if unicodedata.category(c) != 'Mn'
    )

remove_accents('élève préparé')
# 'eleve prepare'
```

Chapitre 4 - Stopwords et stemming

4.1 C'est quoi un stopwords ?

Un **stopword** est un mot tres frequent mais qui apporte **peu d'information semantique** : 'le', 'la', 'de', 'et', 'a', 'pour', etc.

4.2 Pourquoi les supprimer ?

- + Reduit la taille du vecteur TF-IDF
- + Concentre le signal sur les mots importants
- + Les phrases comparees ont la meme structure (pas biaisé par 'le' vs 'la')

4.3 Notre liste de stopwords

```
STOPWORDS_FR = {
    'le', 'la', 'les', 'un', 'une', 'des', 'du', 'de',
    'et', 'ou', 'mais', 'donc', 'or', 'car',
    'je', 'tu', 'il', 'elle', 'nous', 'vous', 'ils',
    'ce', 'cet', 'cette', 'ces',
    'qui', 'que', 'quoi', 'dont', 'ou',
    'est', 'sont', 'avoir', 'etre', 'faire',
    'pas', 'ne', 'plus', 'moins', 'tres',
    'pour', 'par', 'avec', 'sans', 'sous', 'sur', 'dans',
    'comment', 'pourquoi', 'quand', 'combien',
    # ... ~110 mots
}

tokens = [t for t in tokens if t not in STOPWORDS_FR and len(t) > 1]
```

4.4 C'est quoi le stemming ?

Le **stemming** reduit un mot a sa **racine**. Exemple : 'mangeait', 'mangions', 'mangerai' -> 'mang'. Ainsi, ces 3 mots sont consideres identiques en NLP.

4.5 Snowball stemmer (NLTK)

```
from nltk.stem.snowball import FrenchStemmer

stemmer = FrenchStemmer()

stemmer.stem('mangeait')      # 'mang'
stemmer.stem('mangerons')     # 'mang'
stemmer.stem('etudiant')      # 'etudi'
stemmer.stem('etudiante')     # 'etudi'
stemmer.stem('etudier')       # 'etudi'

# Tous reduits a 'etudi' = consideres identiques
```

■ Le stemming n'est PAS du dictionnaire (les stems ne sont pas des mots reels). C'est une heuristique qui marche bien en pratique.

Chapitre 5 - TF-IDF en details

5.1 Intuition

TF-IDF = Term Frequency - Inverse Document Frequency. C'est une formule qui mesure l'importance d'un mot dans un document, en fonction de :

- **TF** : frequence du mot dans CE document (plus il apparait, plus important)
- **IDF** : rarete du mot dans TOUS les documents (plus c'est rare, plus c'est discriminant)

■ **ANALOGIE** : Pour identifier un livre : **TF** = combien de fois 'sorcier' apparait dans Harry Potter. **IDF** = combien de livres parlent de 'sorcier' (rare). 'le' a un TF eleve PARTOUT donc IDF faible -> on l'ignore. 'sorcier' a un TF eleve SEULEMENT dans HP -> tres discriminant.

5.2 Formules mathematiques

$$TF(t, d) = (\text{nombre de fois ou } t \text{ apparait dans } d) / (\text{nb total de mots dans } d)$$

$$IDF(t) = \log(N / df(t))$$

ou N = nombre total de documents

$df(t)$ = nombre de docs qui contiennent t

$$TF-IDF(t, d) = TF(t, d) * IDF(t)$$

5.3 Exemple chiffre

Documents :

D1 = "comment inscription master IADS"
 D2 = "comment inscription master MIAGE"
 D3 = "bonjour comment allez vous"

Pour $t = \text{'master'}$:

$TF(\text{master}, D1) = 1/4 = 0.25$
 $TF(\text{master}, D2) = 1/4 = 0.25$
 $TF(\text{master}, D3) = 0/4 = 0$

$IDF(\text{master}) = \log(3/2) = 0.176$

$TF-IDF(\text{master}, D1) = 0.25 * 0.176 = 0.044$

$TF-IDF(\text{master}, D2) = 0.25 * 0.176 = 0.044$

$TF-IDF(\text{master}, D3) = 0$

Pour $t = \text{'iads'}$:

$TF(\text{iads}, D1) = 1/4 = 0.25$

$IDF(\text{iads}) = \log(3/1) = 0.477$

$TF-IDF(\text{iads}, D1) = 0.25 * 0.477 = 0.119$

'iads' est plus discriminant que 'master' (TF-IDF plus eleve)

5.4 Implementation avec scikit-learn

```
from sklearn.feature_extraction.text import TfidfVectorizer
```



```
documents = [
    "comment inscription master IADS",
    "comment inscription master MIAGE",
    "bonjour comment allez vous",
]

vectorizer = TfidfVectorizer(
    ngram_range=(1, 3), # uni + bi + tri-grammes
    min_df=1,           # mot doit apparaitre au moins 1 fois
    sublinear_tf=True,  # ln(1+tf) au lieu de tf brut
)
tfidf_matrix = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())
print(tfidf_matrix.shape) # (3, 12) - 3 docs, 12 features
```

Chapitre 6 - Cosine similarity

6.1 Intuition

Une fois qu'on a vectorise les documents, comment savoir lesquels sont **similaires** ? On calcule l'**angle** entre leurs vecteurs.

■ **ANALOGIE** : Imagine que chaque document est une **fleche** partant de l'origine. Deux fleches qui pointent dans **la meme direction** = documents similaires. Deux fleches **perpendiculaires** = documents sans rapport. **Opposees** = contraires.

6.2 Formule

$$\text{cosine}(A, B) = (A \cdot B) / (|A| * |B|)$$

Ou :

$A \cdot B$ = produit scalaire des vecteurs
 $|A|$ = norme du vecteur A

Resultat :

1 = vecteurs identiques (meme direction)
 0 = vecteurs perpendiculaires (aucune similitude)
 -1 = vecteurs opposes (rare avec TF-IDF qui est ≥ 0)

6.3 Pourquoi cosine et pas distance euclidienne ?

La distance euclidienne ($|A - B|$) est sensible a la **longueur** des documents. Un long document n'est PAS forcément different d'un court avec le meme contenu. La cosine ignore la magnitude et compare juste la **direction** (= le contenu semantique).

6.4 Implementation

```
from sklearn.metrics.pairwise import cosine_similarity

# Vectoriser la query utilisateur
query = "je veux faire master IADS"
query_vec = vectorizer.transform([query])

# Calculer la similarite avec tous les documents d'entrainement
sims = cosine_similarity(query_vec, tfidf_matrix)[0]

# sims est un array : [sim_avec_D1, sim_avec_D2, sim_avec_D3]
print(sims)
# [0.85, 0.42, 0.05] -> D1 est la plus pertinente
```

6.5 Top-K candidats

```
import numpy as np

# Trier les indices par similarite decroissante
top_indices = np.argsort(sims)[::-1][:5]
```

```
# top_indices = [0, 1, 2, ...] (indices de documents)
for idx in top_indices:
    score = sims[idx]
    intent_tag = pattern_to_intent[idx]
    print(f'{intent_tag}: {score:.3f}')
```

Chapitre 7 - n-grammes

7.1 C'est quoi un n-gramme ?

Un **n-gramme** est une **sequence de N tokens consecutifs**. Au lieu de considerer les mots isolement, on considere aussi leurs combinaisons.

7.2 Exemple

```
Phrase : 'comment m_inscrire en master IADS'

Unigrammes (1-grammes) :
['comment', 'minscrire', 'master', 'IADS']

Bigrammes (2-grammes) :
['comment minscrire', 'minscrire master', 'master IADS']

Trigrammes (3-grammes) :
['comment minscrire master', 'minscrire master IADS']
```

7.3 Pourquoi utiles ?

Les n-grammes capturent les **expressions composees** qui ont un sens different de leurs mots isoles.

- 'pomme de terre' n'est PAS 'pomme' + 'de' + 'terre' separees
- 'emploi du temps' est une expression specifique
- 'master IADS' n'est PAS 'master' generique + 'IADS'

7.4 Configuration scikit-learn

```
vectorizer = TfidfVectorizer(
    ngram_range=(1, 3),    # uni + bi + tri-grammes
)

# Pour 'master IADS' avec ngram_range=(1,3) :
# - unigrammes : 'master', 'IADS'
# - bigramme   : 'master IADS'
# - trigrammes : aucun (trop court)
```

[★] Notre projet utilise `ngram_range=(1, 3)` qui capture beaucoup d'expressions FSBM specifiques : 'emploi du temps', 'master IADS', 'kifash ntsajjel', etc.

Chapitre 8 - Detection de langue

8.1 Pourquoi detecter la langue ?

Si l'utilisateur ecrit en darija, on veut chercher dans les patterns darija et repondre en darija. Si en francais, idem. La detection doit se faire **avant** le matching.

8.2 Notre algorithme (hybride)

On combine 3 strategies en cascade :

- 1. Caracteres arabes.** Si le message contient des caracteres Unicode arabes -> darija/arabe.
- 2. Chiffres-lettres.** Les chiffres 3, 7, 9, 8 dans des mots latins sont des marqueurs darija forts. Ex: '3am' (annee), '7biba' (mon amour), '9rib' (proche).
- 3. Score lexical.** Comptage des mots-cles distinctifs par langue. ~110 mots FR, 100+ EN, 80+ darija.

8.3 Mots-cles distinctifs

```
DARIJA_KEYWORDS = {
    'ana', 'nta', 'nti', 'wesh', 'wach',
    'kifash', 'kif', 'ach', 'achno', 'chno',
    'fin', 'imta', 'bghit', 'kandir',
    'khdma', 'khassek', 'shokran', 'salam',
    'labas', 'bslama', 'marhba', 'mzyan',
    'kanstress', 'kanhass', 'ma3andich',
    # ... ~80 mots
}

FRENCH_KEYWORDS = {
    'le', 'la', 'je', 'tu', 'comment', 'pourquoi',
    'inscription', 'master', 'filiere', 'examen',
    # ... ~110 mots
}

ENGLISH_KEYWORDS = {
    'the', 'is', 'are', 'how', 'what',
    'master', 'enrollment', 'student',
    # ... ~100 mots
}
```

8.4 Algorithme final

```
def detect_language(text: str, default='fr') -> str:
    # 1. Caracteres arabes ?
    if has_arabic_script(text):
        return 'darija'

    tokens = tokenize(text.lower())
    tokens_set = set(tokens)

    # 2. Chiffres-lettres darija ?
```

```
if has_darija_numerals(tokens):  
    return 'darija'  
  
# 3. Score lexical  
fr = len(tokens_set & FRENCH_KEYWORDS)  
en = len(tokens_set & ENGLISH_KEYWORDS)  
dar = len(tokens_set & DARIJA_KEYWORDS)  
  
total = fr + en + dar  
if total == 0:  
    return default  
  
scores = {'fr': fr, 'en': en, 'darija': dar}  
return max(scores, key=scores.get)
```

[✓] Taux de detection : **14/14 tests reussis** (100%). Fonctionne sans aucune dependance externe (pas langdetect, pas cld3) - tout en Python pur.

Chapitre 9 - Multilingue (FR/EN/Darija)

9.1 Strategie : 3 modeles paralleles

Au lieu d'avoir un seul modele qui melange les 3 langues, on entraine **3 modeles TF-IDF separes**, un par langue. Cela donne :

- + Vocabulaire distinct par langue (pas de pollution)
- + Meilleur matching dans la langue native
- + Possibilite d'enrichir une langue sans toucher les autres

9.2 Structure du dataset

```
// faq_dataset.json
{
  "version": "3.2.0",
  "languages": ["fr", "en", "darija"],
  "intents": [
    {
      "tag": "inscription",
      "patterns": {
        "fr": ["Comment m'inscrire", "Procedure inscription", ...],
        "en": ["How to enroll", "Registration procedure", ...],
        "darija": ["Kifash ntsajjel", "Bghit ntsajjel f FSBM", ...]
      },
      "responses": {
        "fr": ["Pour s'inscrire a la FSBM : ..."],
        "en": ["To enroll at FSBM : ..."],
        "darija": ["Bach ttsajjel f FSBM : ..."]
      }
    },
    // ...
  ]
}
```

9.3 Entrainement multilingue

```
class MultilingualClassifier:
    def train(self):
        # Un vectorizer par langue
        self.vectorizers = {}          # {'fr': ..., 'en': ..., 'darija': ...}
        self.tfidf_matrices = {}
        self.pattern_to_intent = {}

        for lang in ['fr', 'en', 'darija']:
            patterns = []
            tags = []

            for intent in self.intents:
                pats = intent['patterns'].get(lang, [])
                for p in pats:
                    patterns.append(self.preprocess(p))
                    tags.append(intent['tag'])
```

```
if patterns:
    vec = TfidfVectorizer(ngram_range=(1, 3), sublinear_tf=True)
    matrix = vec.fit_transform(patterns)
    self.vectorizers[lang] = vec
    self.tfidf_matrices[lang] = matrix
    self.pattern_to_intent[lang] = tags
```

9.4 Statistiques du modele entraine

Langue	Patterns	Features TF-IDF	Intents
FR	188	341	27
EN	186	565	27
Darija	461	1507	27
TOTAL	835	~2400	28

Chapitre 10 - Memoire conversationnelle

10.1 Pourquoi une memoire ?

Sans memoire, chaque message est traite isolement. Avec memoire, le bot peut :

- Se souvenir du nom de l'utilisateur
- Connaitre son genre (khoya/khti)
- Suivre la conversation (resolution de pronoms : 'pour ça' -> 'pour le master IADS')
- Adapter le ton selon les tours precedents

10.2 Notre architecture

```
class SessionContext:
    session_id: str
    user_id: Optional[int]
    started_at: datetime
    history: deque[Turn]          # max 20 derniers tours
    user_context: dict           # {'filier', 'annee', 'gender', 'name'}
    total_turns: int

class Turn:
    user_message: str
    bot_response: str
    intent: str
    confidence: float
    timestamp: datetime
```

10.3 Singleton ConversationMemory

```
class ConversationMemory:
    _instance = None
    _sessions: dict[str, SessionContext] = {}

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def get_or_create(self, session_id, user_id=None):
        if session_id not in self._sessions:
            self._sessions[session_id] = SessionContext(
                session_id=session_id,
                history=deque(maxlen=20)
            )
        return self._sessions[session_id]

    def add_turn(self, session_id, user_msg, bot_resp, intent, conf, user_id=None):
        ctx = self.get_or_create(session_id, user_id)
        ctx.history.append(Turn(user_msg, bot_resp, intent, conf))
        self._extract_context(ctx, user_msg, intent)

memory = ConversationMemory() # singleton global
```

10.4 Extraction automatique de contexte

```
def _extract_context(ctx: SessionContext, message: str, intent: str):
    msg = message.lower()

    # Filieres mentionnees
    for code, keywords in {
        'SMI': ['smi', 'sciences mathematiques et informatique'],
        'DI': ['di', 'developpement informatique'],
        # ...
    }.items():
        if any(k in msg for k in keywords):
            ctx.user_context['filierere'] = code
            break

    # Annee d'etudes
    for n, kw in [(1, ['s1', 's2', 'premiere annee', 'L1']),
                  (2, ['s3', 's4', 'deuxieme annee', 'L2']),
                  (3, ['s5', 's6', 'troisieme annee', 'L3'])]:
        if any(k in msg for k in kw):
            ctx.user_context['annee'] = n
```

Chapitre 11 - Personnalisation (genre/nom)

11.1 Detection du genre

```
GENDER_HINTS_F = [
    r'\bana\s+bnt\b',          # Darija
    r'\bana\s+fata\b',
    r'je\s+suis\s+une?\s+fille', # FR
    r'je\s+suis\s+etudiante',
    r"i\s+am\s+a?\s*(girl|female)", # EN
]

GENDER_HINTS_M = [
    r'\bana\s+wld\b',
    r'\bana\s+rajl\b',
    r'je\s+suis\s+un\s+garcon',
    r"i\s+am\s+a?\s*(boy|male|man)",
]

def detect_gender(message: str) -> Optional[Gender]:
    lower = message.lower()
    for pat in GENDER_HINTS_F:
        if re.search(pat, lower):
            return 'F'
    for pat in GENDER_HINTS_M:
        if re.search(pat, lower):
            return 'M'
    return None
```

11.2 Substitution dans les reponses

Les reponses pre-ecrites contiennent des **placeholders** qui sont remplaces selon le genre detecte :

```
Template : 'Salam {voc}, ach {dayer_dayra} ? Je suis content de te connaitre.'

Pour un homme (gender=M) :
-> 'Salam khoya, ach dayer ?'

Pour une femme (gender=F) :
-> 'Salam khti, ach dayra ?'

Genre inconnu :
-> 'Salam sahbi, ach dayer/dayra ?'
```

11.3 Code de substitution

```
def personalize_response(text, gender=None, name=None, lang='darija'):
    if gender == 'F':
        subs = {
            '{voc}': 'khti',
            '{dayer_dayra}': 'dayra',
            '{stresse}': 'stressee',
            '{name}': name or '',
        }
    elif gender == 'M':
        subs = {
```

```
        '{voc}': 'khoya',
        '{dayer_dayra}': 'dayer',
        '{stresse}': 'stresse',
        '{name}': name or '',
    }
else:
    subs = {
        '{voc}': 'sahbi',
        '{dayer_dayra}': 'dayer/dayra',
        '{stresse}': 'stresse(e)',
        '{name}': '',
    }

    for placeholder, value in subs.items():
        text = text.replace(placeholder, value)
    return text
```

Chapitre 12 - LLM (LLaMA 3 + Groq)

12.1 Limites du TF-IDF

Le TF-IDF est parfait pour des FAQs structurees, mais limite pour :

- Questions formulees avec des mots **inconnus** du dataset
- Questions combinant **plusieurs sujets** ('master IADS + bourse + logement')
- Conversations longues avec **resolution d'ambiguites**
- Reponses **creatives** ou tres personnalisees

12.2 Solution : LLM (LLaMA 3.3)

Un **LLM (Large Language Model)** comme LLaMA 3 peut comprendre le sens semantique et generer des reponses uniques. Pour notre PFE on utilise **Groq** qui heberge LLaMA 3 gratuitement et tres rapidement (~150ms).

12.3 Client Groq

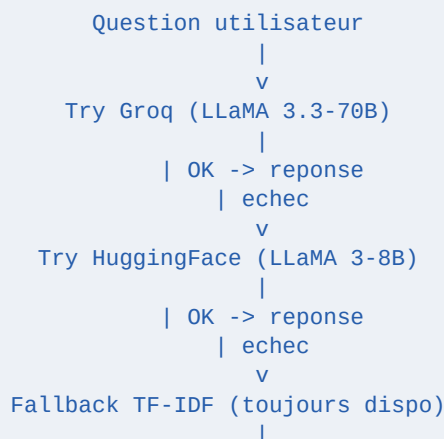
```
from groq import Groq

client = Groq(api_key='gsk_xxx')

completion = client.chat.completions.create(
    model='llama-3.3-70b-versatile',
    messages=[
        {'role': 'system', 'content': 'Tu es un assistant FSBM. Reponds en francais.'},
        {'role': 'user', 'content': 'Quelles filieres a la FSBM ?'},
    ],
    temperature=0.7,
    max_tokens=1024,
)

print(completion.choices[0].message.content)
```

12.4 Architecture en cascade (fallback)



v
Reponse pre-ecrite garantie

■ Cette cascade garantit **100% de disponibilite**. Le chatbot ne tombe JAMAIS. Au pire, il revient au comportement de la v1 (TF-IDF).

Chapitre 13 - RAG = retrieval + generation

13.1 Le probleme du LLM 'pur'

Un LLM 'pur' peut HALLUCINER : inventer des informations qui ont l'air vraies mais sont fausses. Dangereux pour une fac (numero de tel imagine, date d'examen fausse).

13.2 Solution : RAG

RAG = Retrieval-Augmented Generation. On combine 2 systemes :

1. **RETRIEVAL.** On cherche dans notre dataset FSBM les passages les plus pertinents pour la question.
2. **AUGMENTED.** On injecte ces passages dans le prompt du LLM.
3. **GENERATION.** Le LLM repond en se basant sur ces passages, donc SANS inventer.

■ **ANALOGIE :** Sans RAG = etudiant qui repond de tete (parfois faux). Avec RAG = etudiant qui ouvre son cahier de cours specifique a la FSBM et cite les passages exacts. La qualite est bien meilleure.

13.3 Notre RAG : retrieval avec TF-IDF

On REUTILISE notre classifieur TF-IDF comme retriever. Pas besoin d'embeddings neuronaux.

```
class RAGRetriever:
    def __init__(self, classifieur):
        self.classifieur = classifieur # MultilingualClassifier deja entraine

    def retrieve(self, query, lang='fr', top_k=3):
        # Pretraitement comme dans predict()
        processed = self.classifieur.preprocessor.preprocess(query)

        # Vectorisation TF-IDF dans la bonne langue
        vec = self.classifieur.vectorizers[lang].transform([processed])

        # Similarite cosinus contre tous les patterns
        sims = cosine_similarity(vec,
                                self.classifieur.tfidf_matrices[lang])[0]

        # Top-K candidats (dedoublonnes par intent)
        # ...

        return [{
            'tag': intent_tag,
            'score': score,
            'patterns': intent.patterns[lang][:5],
            'reference_response': intent.responses[lang][0],
        }, ...]
```

13.4 Construction du prompt RAG

```
def build_rag_prompt(user_message, contexts, lang='fr',
                    gender=None, name=None):
    system = SYSTEM_PROMPTS[lang] # role officiel FSBM

    if contexts:
        system += '\n\n=== CONTEXTE FSBM ===\n'
        for i, c in enumerate(contexts, 1):
            system += f'Passage #{i} (sujet: {c.tag}, score: {c.score})\n'
            system += f'Exemples : {c.patterns[:3]}\n'
            system += f'Information : {c.reference_response}\n\n'
        system += '=== FIN CONTEXTE ==='

    # Info personnelle (genre/nom)
    if gender or name:
        system += f'\n[INFO : nom={name}, genre={gender}]'

    return system, user_message
```

13.5 Exemple de prompt RAG complet

```
SYSTEM :
Tu es l'assistant officiel de la FSBM.
Reponds UNIQUEMENT en utilisant le contexte ci-dessous.
Si l'info n'est pas dans le contexte, dis 'Je ne sais pas'.

=== CONTEXTE FSBM ===
Passage #1 (sujet: master_iads, score: 0.91)
Master IADS - IA & Data Science. 2 ans, 25 places.
Programme : ML, Deep Learning, NLP, Vision, Big Data...

Passage #2 (sujet: bourses, score: 0.65)
Bourse ONOUCS : revenus famille + distance. ~3500 DH/an.
Demande sur www.onoucs.ma en juillet-aout.
=== FIN CONTEXTE ===

USER :
Je veux faire le master IADS, est-ce qu'il y a une bourse ?

ASSISTANT (genere par LLaMA) :
Oui, vous pouvez postuler a la fois au master IADS et a la bourse ONOUCS.
Le master IADS est selectif (25 places). La bourse ONOUCS vous donnerait
~3500 DH/an si vous remplissez les criteres (revenus + distance).
```


Chapitre 14 - Embeddings (vectorielle moderne)

14.1 Limite du TF-IDF

TF-IDF compare les **mots EXACTS**. Si l'utilisateur dit 'chat' et notre pattern dit 'feline', le score est 0. C'est limitant.

14.2 Solution : Embeddings neuronaux

Un **embedding** est un vecteur de nombres qui represente le **sens** d'un mot ou d'une phrase. Les mots de sens proche ont des embeddings proches (meme s'ils sont differents lexicalement).

```
embedding('chat')    = [0.23, -0.45, 0.81, ..., 0.34]
embedding('feline')   = [0.21, -0.43, 0.79, ..., 0.32] # tres proche !
embedding('meteo')    = [-0.89, 0.12, -0.34, ..., -0.23] # tres different
```

14.3 Modeles d'embedding populaires

- **OpenAI text-embedding-3** : commercial, dim 1536-3072
- **Sentence-BERT** : open source, dim 384-768
- **BGE-M3** : open source multilingue, dim 1024
- **Multilingual-E5** : open source, 100 langues

14.4 Quand passer du TF-IDF aux embeddings ?

Critere	TF-IDF (notre v1)	Embeddings (v2 future)
Dimensions	~1500	384-3072
Semantique	Aucune	Excellente
Vitesse	Tres rapide (<10ms)	Lent (CPU) ou GPU
Cout	Gratuit	Gratuit en local
Adapte a	28 intents structures	Documents longs varies
Notre PFE	Suffisant	Phase 2 si scale

[★] Pour notre PFE (28 intents), TF-IDF suffit largement et est tres rapide. Si on passait a 10 000 documents (Phase 2), on basculerait sur BGE-M3 ou Sentence-BERT.

Chapitre 15 - Conclusion

15.1 Recap des concepts

- + NLP = traitement automatique du langage
- + Pipeline : detection langue -> preprocessing -> vectorisation -> matching
- + Tokenisation = decoupage en mots
- + Stopwords = mots peu informatifs a enlever
- + Stemming = reduction a la racine
- + TF-IDF = importance d'un mot (frequence + rarete)
- + Cosine similarity = proximite entre vecteurs
- + n-grammes = capture expressions composees
- + Detection langue hybride (caracteres + mots-cles)
- + Multilingue = 3 modeles TF-IDF separes
- + Memoire = historique + contexte utilisateur
- + Personnalisation = substitution placeholders selon genre/nom
- + LLM = LLaMA 3 via Groq pour reponses naturelles
- + RAG = retrieval + augmentation prompt + generation
- + Embeddings = vectorisation semantique moderne (Phase 2)

15.2 Pour aller plus loin

- PDF 09 - workflow complet d'un message dans le pipeline
- PDF 10 - guide soutenance avec Q/R sur l'IA
- Le PDF Guide_IA_Pedagogique existant (192 KB) qui detaille Groq, LLaMA, RAG